

Hands-On-Learning: : Build a Production Monitoring, Optimization, and Cost Control Workflow

Learning Objectives

By the end of this activity, learners will be able to:

- Instrument an API with Prometheus metrics to monitor latency, throughput, and error rates.
- Build a Grafana dashboard that visualizes key SLO indicators.
- Apply caching and database optimization strategies to improve performance.



- Calculate cost per request and evaluate the impact of cost optimization techniques.
- Define RTO/RPO targets for a reliable production operations plan.



Description

In this hands-on practice, you will design a streamlined production operations workflow that includes monitoring, performance optimization, cost reduction, and basic reliability planning for a large-scale API.

Structure of the Activity

Title: Build a Production Monitoring, Optimization, and Cost Control Workflow

Dataset Information / Resources Required:

Attached Input Documents:

- **Lab Instructions File:** /mnt/data/m3_lab_instructions.docx
- **Starter Code File:** /mnt/data/m3_lab_starter.docx
- **Solution File:** /mnt/data/m3_lab_solution.docx
- **Dataset File:**
/mnt/data/m3_design_operations_platform_dataset_readme.md

Dataset Notes: This practice uses **synthetic monitoring and cost metrics**, including latency percentiles, cache hit rate, cost-per-request data, and availability indicators. Data is conceptual and does not require preprocessing.

Tools Used:

- Python 3.x
- Prometheus client library (prometheus_client)
- Grafana dashboard API (grafana_api)
- Redis cache (conceptual use)
- Basic database utilities (indexing, performance concepts)

Setup Requirements

A Python execution environment (local or notebook).

Prometheus and Grafana installed (or conceptually assumed for the exercise).

Access to the provided files in your working directory.

Basic familiarity with API operations and SRE concepts.

Scenario

You are a platform engineer at **GlobalCorp**, responsible for supporting an LLM API used by millions of customers.

Recent performance spikes, cost overruns, and reliability issues have drawn leadership attention. The operations team needs a unified workflow that provides:

- real-time observability into API behavior,
- optimized performance during peak traffic, and
- controlled infrastructure spending without breaking SLAs.

Your task is to design a compact operations workflow addressing these three mission-critical needs.

Objective

Create a monitoring and optimization workflow that instruments the API, visualizes key performance indicators, improves efficiency using caching and indexing, reduces cost per request, and establishes RTO/RPO guidelines for platform reliability.



Getting Started:

- 1 Review the attached lab instructions and starter code files to understand the original workflow.
- 2 Ensure all files are placed in your working directory for reference (no execution dependency).
- 3 Confirm that your Python environment can import standard libraries and pseudo-modules used in examples.

Activity 1: Instrument Key Metrics and Build Dashboards

Objective

Instrument essential API metrics and create a Grafana dashboard that visualizes request rate, latency, and error behavior.



Input/Data:

→ Reference Files:

→ *m3_lab_starter.docx*

→ *m3_lab_instructions.docx*

→ **Metrics Data:** Synthetic latency, error rate, and throughput metrics.

→ Required imports (conceptual only):

```
from prometheus_client import Counter, Histogram, Gauge  
from grafana_api import create_dashboar
```

Steps to Complete

Step 1: Identify core metrics that reflect API behavior, including request rate, latency percentiles, and error counts.

Step 2: Create Prometheus metric definitions using counters, histograms, and gauges to represent these indicators.

Step 3: Add an additional metric representing cost per request or token usage to visualize operational cost trends.

Step 4: Build a simple Grafana dashboard configuration that displays the metrics using time-series panels.

Step 5: Add queries to track p95 latency, request throughput, and error rates so students can observe performance patterns.

Expected Output

Dashboard configuration that displays real-time trends for request rate, latency, and errors.

Metric objects defined and ready for use in a monitoring system.

A basic awareness of how dashboard panels map to Prometheus queries.

Tips

Ensure metric names follow Prometheus conventions (lowercase, underscores).

If latency queries return empty results, confirm histogram buckets exist.

Avoid overcrowding the dashboard—focus only on essential SLO indicators.

Activity 2: Improve Performance Using Caching and Database Optimization

Objective

Apply caching and basic database optimization methods to reduce latency and improve overall system responsiveness.



Input/Data:

Reference Files:

m3_lab_starter.docx

m3_lab_solution.docx

Required imports (conceptual):

from caching import setup_redis_cache, warm_cache
from db_optimization import analyze_queries, add_indexes

Data from previous activity:

Observed latency and error metrics (conceptual)

Steps to Complete

Step 1: Analyze which requests or operations produce slow responses. Identify at least two patterns that can benefit from caching.

Step 2: Define a caching strategy that includes normalized keys and appropriate TTL values to keep hot data readily available.

Step 3: Simulate a cache warm-up process for frequent queries and estimate its impact on hit rate.

Step 4: Examine common database queries and propose indexing on fields frequently used as filters or joins.

Step 5: Summarize performance improvements by comparing expected response speed before and after caching and indexing.

Expected Output

Description of caching strategy with TTL and key normalization.

List of recommended indexes and their intended effect.

Qualitative improvement indicator (e.g., “expected latency reduction,” “higher cache hit rate”).

Tips

Avoid caching responses that change very frequently.

Improper indexing can worsen performance if applied to low-selectivity fields.

Cache warm-up simulations should prioritize the top repeated queries.

Activity 3: Calculate Cost Efficiency and Apply Optimization Techniques

Objective

Evaluate cost per request and apply cost optimization techniques such as model tiering and autoscaling adjustments.



Input/Data:

Reference Files:

m3_lab_instructions.doc

o m3_lab_solution.docx

Dataset File:

m3_design_operations_platform_dataset_readme.md

Required imports (conceptual):

```
from cost_optimization import setup_llm_router, configure_autoscaling
```

Steps to Complete

Step 1: Identify the main components contributing to operational cost (compute, storage, API calls, token usage).

Step 2: Estimate the baseline cost per request using synthetic metrics provided in the dataset notes.

Step 3: Propose a model-tiering strategy that uses lightweight models for simple queries and higher-tier models only when required.

Step 4: Define autoscaling adjustments that reduce on-demand compute usage and increase spot instance utilization.

Step 5: Recalculate the cost per request after applying optimization strategies and estimate potential monthly savings.

Expected Output

A table or short note comparing baseline vs. optimized cost per request.

Clear explanation of how model tiering reduces unnecessary expenditure.

Estimated monthly savings based on scaled request volume.

Tips

Make sure to base cost reductions on realistic request volumes.

Avoid relying solely on spot instances—they can be reclaimed without warning.

Always maintain minimum resources for SLA compliance.

Activity 4: Establish Reliability Targets Using RTO and RPO

Objective

Define high-level reliability goals by creating RTO and RPO targets for critical operations.



Input/Data:

Reference Files:

m3_lab_instructions.docx

Required imports (conceptual):

```
from disaster_recovery import define_rto_rpo
```

Steps to Complete

Step 1: List the system's critical operations such as write-path APIs, user session state, and billing workflows.

Step 2: Assign RTO (maximum acceptable downtime) and RPO (acceptable data loss window) values to each critical operation.

Step 3: Prioritize operations based on business impact and customer sensitivity.

Step 4: Summarize these reliability targets into a compact DR readiness table.

Expected Output

A simple RTO/RPO table listing critical operations and target values.

Clear understanding of which systems require zero data loss and near-zero downtime.

Foundation for a future full disaster recovery plan.

Tips

RTO values less than five minutes require automation rather than manual intervention.

RPO of zero requires synchronous replication or strong write durability.

Avoid overcommitting unrealistic RTO/RPO targets that infrastructure cannot meet.

Compile Your Findings

Requirement 1: Monitoring Instrumentation and Dashboard Setup

Description: Demonstrate correct implementation of API monitoring through defined Prometheus metrics and a functional Grafana dashboard layout.

Include:

Specific Code / Output Required

Metric definitions for request rate, latency histogram, error count, and cost-per-request gauge.

A dashboard configuration containing at least three panels (latency, request rate, error percentage).

Evidence of Understanding

- A short explanation of why p95 latency, error percentage, and throughput are critical for SLO monitoring.
- A description of how each selected metric helps diagnose performance issues.

Requirement 2: Performance and Cost Optimization Workflow

Description: Show the optimization plan that improves latency and reduces cost per request through caching, indexing, model tiering, and autoscaling strategy.

Include:

Implementation Details

- A defined caching strategy (TTL, key normalization, top-query warming).
- A list of recommended database indexes and the reason each improves performance.
- A model tiering strategy that assigns lightweight and heavy models appropriately.
- A conceptual autoscaling configuration (baseline instances + spot capacity).

Results / Metrics

- Baseline vs. optimized cost per request estimation.
- Expected change in latency (qualitative or estimated).
- Expected cache hit rate improvement based on warmup strategy.

Requirement 3: Reliability Analysis and Business Implications

Description: Provide reliability targets (RTO and RPO) for critical operations and interpret the business impact of the monitoring and optimization workflow.

Include:

→ **Analysis / Interpretation**

→ A table listing RTO and RPO values for core operations (writes, reads, billing, user sessions).

→ Explanation of which systems require zero-data-loss or near-zero downtime.

Business Implications

- Short justification of how enhanced monitoring improves incident response time.
- How caching and indexing directly reduce operational costs.
- How model tiering and autoscaling contribute to predictable monthly spending.
- How DR targets align with SLA obligations and customer trust.

Summary

This activity teaches **production monitoring, performance optimization, cost reduction, and reliability planning** through practical implementation.

You've learned how to **instrument key API metrics, design dashboards, optimize caching and indexing, calculate cost improvements, and define RTO/RPO targets**—skills directly applicable to **operating large-scale systems in real enterprise environments**.

Resources

Dataset: /mnt/data/m3_design_operations_platform_dataset_readme.md

Documentation:

- Prometheus Metrics Reference
- Grafana Dashboard & Query Guide
- Redis Caching Concepts
- Basic Indexing and Query Optimization Concepts

Resources

None required

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.